

Fresh verification results

Verified Docker Desktop run | 2026-09-23 | UTC and Pakistan Standard Time (UTC+05:00)

Fresh Docker verification

Run date: 23 September 2026. Raw evidence: docs/evidence/latest/. Existing .env and named volumes were preserved. The complete stack was rebuilt and restarted; no application redesign was required.

Tests and business behavior

The PostgreSQL-backed Docker suite passed **26 tests in 3.46 seconds**, without skips. Fresh concurrency: **1 winner and 19 conflicts**. Smoke verified room listing, booking creation (201), lookup, cancellation and a clean overlapping-booking rejection (409). Health, readiness, Swagger/OpenAPI and metric endpoints were checked. Readiness exercises PostgreSQL connectivity.

Metrics and dashboards

Prometheus app, node and self targets are UP. The optional cardinality target is UP when the final experiment is left running. Counter, Gauge, Histogram and Summary declarations and samples were checked. All **3 dashboards, 40 panels and 46 expressions** returned real data at the fresh baseline timestamp. This covers throughput, server/client tails, error rate, in-flight requests, booking/conflict metrics, DB duration, CPU, memory, filesystems and network. Grafana's fresh baseline was also opened in a real browser; its visible legends showed real rates/latencies and zero error fraction. Empty post-restart labelled metrics before business traffic were investigated; the historical verifier now applies its requested timestamp consistently to metric-presence checks.

Fresh logging evidence

FastAPI JSONL -> Filebeat -> Elasticsearch -> Kibana was verified by the live stack script. The stored document preserves timestamp, service, severity, message, request_id, method, route, numeric status_code and duration_ms. The data view uses @timestamp.

Request ID: verify-0c74ea91-41be-4bab-864b-80388a2d07d4. Exact KQL: request_id :
"verify-0c74ea91-41be-4bab-864b-80388a2d07d4".

Actual event: **2026-09-23T04:19:55.674Z**, POST /bookings, HTTP **201**, **7.573 ms**. Screenshot range: UTC: 2026-09-23 04:19:25.674 to 2026-09-23 04:20:25.674; Pakistan time: 2026-09-23 09:19:25.674 to 2026-09-23 09:20:25.674. The actual stored document and ILM policy attachment are in stack-verification.json; no screenshot has been invented.

Fresh performance measurements

Run	Requests/s	Client p95 / p99	Unexpected
Fresh standalone baseline	159.35	48.15 / 66.57 ms	0.00%
Anomaly baseline	154.92	52.11 / 70.69 ms	0.00%
Anomaly fault	66.23	516.32 / 526.49 ms	0.00%
Anomaly recovery	157.71	53.49 / 71.44 ms	0.00%

The standalone baseline command ran from UTC: 2026-09-23 04:20:01.784 to 2026-09-23 04:22:04.464; Pakistan time: 2026-09-23 09:20:01.784 to 2026-09-23 09:22:04.464. All profiles use 10 virtual users for two minutes and the existing business workflow. k6 totals include setup readiness; server business metrics exclude probes. Rates and percentiles come from the saved summaries, not estimates or old runs.

Anomaly and recovery

The existing experiment script runs baseline, 500 ms delay every fifth business request, then fault-free recovery. Its 310-second inter-phase cooldowns prevent five-minute dashboard windows from mixing phases. Exact command boundaries are:

Phase	UTC	Pakistan time (UTC+05:00)
baseline	2026-09-23 04:22:08.939 to 2026-09-23 04:24:11.191	2026-09-23 09:22:08.939 to 2026-09-23 09:24:11.191
fault	2026-09-23 04:29:25.313 to 2026-09-23 04:31:28.704	2026-09-23 09:29:25.313 to 2026-09-23 09:31:28.704
recovery	2026-09-23 04:36:43.105 to 2026-09-23 04:38:46.082	2026-09-23 09:36:43.105 to 2026-09-23 09:38:46.082

Fault logs in baseline/fault/recovery: **0 / 1606 / 0**. Matching warning/completion documents are preserved in `anomaly-logs.json`. Recovery removed the slowdown and the final app configuration has fault injection **OFF**. Small baseline/recovery differences remain possible on the shared machine.

Cardinality

100 accepted calls in each controlled mode produced **100 -> 1 current value series**. Query: `count(demo_requests_total{job="cardinality-demo"})`. High observation: UTC: 2026-09-23 04:39:46.493 to 2026-09-23 04:39:46.493; Pakistan time: 2026-09-23 09:39:46.493 to 2026-09-23 09:39:46.493. Low observation: UTC: 2026-09-23 04:39:55.419 to 2026-09-23 04:39:55.419; Pakistan time: 2026-09-23 09:39:55.419 to 2026-09-23 09:39:55.419. Each mode's total was 100; the low-mode 101st call returned 429. The low mode remains running and earlier high samples remain historical.

Final state and limitations

All eight required containers are running and healthy at the final verification. Fault injection is disabled. The cardinality exporter remains healthy in low mode. All four Prometheus targets are UP. Kibana reports all services/plugins available and Task Manager healthy. Retained logs include transient Task Manager degradation during host timer discontinuities and expired browser-session warnings; current service and authenticated datasource checks pass. A post-restart verification race was fixed by waiting up to 30 seconds for Prometheus to scrape newly created histogram labels. The retried end-to-end logging check passed.

This is a local single-worker demonstration on a shared Windows/Docker Desktop machine. Node Exporter measures Docker Linux/WSL2 resources and its visible mounts/network namespace, not native Windows telemetry. Quantiles from histogram buckets are estimates; k6 client percentiles measure a different scope. Deliberate 409 conflicts are not server failures. Closed-loop throughput is not an independently established production capacity. Seven-day expiry was not observed. Historical k6 panels may be empty at Now because the runner sends stale markers; use the prepared absolute ranges. Native Windows and 21 September results remain historical and are not presented as today's runs. The earlier staged load exposed pool saturation at 200 users; that staged experiment was not rerun in this refresh.

All 15 user-captured screenshots have been included in the REPORT.pdf evidence appendix. Original PNGs are preserved in screenshots/. Measurements remain from 23 September 2026; this document regeneration does not claim a new runtime test.